# Architecture Redesign: GO Lifecycle Platform v7.0

## 1. Overview

This document describes the comprehensive architectural overhaul of the Hyperledger Fabric–based Guarantee of Origin (GO) issuance, transfer and conversion system. The redesign transforms the project from a fixed-org monolithic prototype into a **tiered, multi-carrier, object-oriented platform** with a full-stack frontend. Versions 6.0 and 7.0 add production hardening and market integration features including cross-registry bridging, IoT device attestation, and an external data oracle.

### Design Principles

- **Tiered network**: Role-based organizations (Issuer, Producer, Consumer) instead of hardcoded org names
- **Multi-carrier**: Extensible beyond electricity/hydrogen — support any energy carrier via a common GO interface (electricity, hydrogen, biogas as of v5.0)
- **Object-oriented chaincode**: Separate asset types (Device, GO, Certificate) into dedicated files with shared base types
- **Contention-free writes**: Deterministic hash-based IDs derived from transaction IDs — no shared state during ID generation (v3.0)
- **Performance-validated**: Architecture verified by Hyperledger Caliper benchmarks across v3.0, v5.0, and v7.0; 100% write success rate at 50 TPS, reads at 2,000 TPS
- **Bug-free**: Fix all 12 identified bugs from the current monolithic chaincode
- **Full-stack**: TypeScript frontend using the Fabric Gateway client API
- **Standards-aligned**: CEN-EN 16325 field validation, EECS energy source codes (v6.0)
- **Cryptographically secure**: 128-bit random commitment salts, ECDSA P-256 device attestation (v6.0/v7.0)
- **Cross-registry**: Bridge contract for GO import/export with external registries (v7.0)
- **Oracle-verified**: External grid data cross-referencing for production claim validation (v7.0)

---

## 2. Repository Structure (New)

```
HLF-GOconversionissuance-JA-MA/
├── README.md
├── ARCHITECTURE.md                    # This file
├── Project-Description/
│
├── chaincode/                         # ── NEW: Restructured Go chaincode ──
│   ├── go.mod
│   ├── go.sum
│   ├── main.go                        # Entrypoint: registers all contracts
│   ├── assets/                        # Asset type definitions
│   │   ├── base.go                    # BaseGO interface & common fields
│   │   ├── device.go                  # Device (SmartMeter, OutputMeter) asset
│   │   ├── electricity_go.go          # ElectricityGO + private details
│   │   ├── hydrogen_go.go             # HydrogenGO + private details + backlog
│   │   ├── certificate.go             # CancellationStatement, ConsumptionDeclaration
│   │   └── counter.go                 # Persistent on-chain counter (replaces in-memory)
│   ├── contracts/                     # Smart contract logic grouped by domain
│   │   ├── issuance.go                # CreateElectricityGO, CreateHydrogenGO
│   │   ├── transfer.go                # TransferGO, TransferGOByAmount
│   │   ├── conversion.go              # Hydrogen backlog, IssuehGO (e→h conversion)
│   │   ├── cancellation.go            # ClaimRenewableAttributes, VerifyCancellation
│   │   ├── query.go                   # All read/query functions
│   │   └── device_mgmt.go             # RegisterDevice, RevokeDevice, ListDevices
│   ├── access/                        # Access control helpers
│   │   ├── roles.go                   # Role-based checks (IsIssuer, IsProducer, etc.)
│   │   ├── abac.go                    # ABAC attribute helpers (getOrgAttr, device attrs)
│   │   └── collections.go             # Collection name resolution by org role
│   └── util/                          # Shared utilities
│       ├── iterator.go                # Generic iterator constructors
│       ├── validate.go                # Input validation helpers
│       └── split.go                   # Proportional splitting algorithm
│
├── collections/                       # ── NEW: Private data collection configs ──
│   └── collection-config.json         # Dynamic role-based collections
│
├── network/                           # ── NEW: Consolidated network setup ──
│   ├── configtx.yaml                  # Channel configuration (tiered roles)
│   ├── docker/                        # Docker Compose files
│   │   ├── docker-compose-orderer.yaml
│   │   ├── docker-compose-issuer.yaml
│   │   ├── docker-compose-producer.yaml   # Template for any producer
│   │   ├── docker-compose-consumer.yaml   # Template for any consumer
│   │   ├── docker-compose-ca.yaml
│   │   └── docker-compose-couchdb.yaml
│   ├── organizations/                 # Crypto material generation
```

```
│   │   ├── fabric-ca/                # CA configs per org-type
│   │   └── cryptogen/                # Alternative: cryptogen configs
│   ├── scripts/                      # Operational scripts
│   │   ├── network-up.sh
│   │   ├── network-down.sh
│   │   ├── create-channel.sh
│   │   ├── deploy-chaincode.sh
│   │   ├── add-org.sh                # ── NEW: Dynamic org onboarding ──
│   │   └── set-anchor-peers.sh
│   └── base.yaml                     # Base peer/orderer config
│
├── application/                      # ── NEW: Full-stack frontend ──
│   ├── backend/                      # Node.js/Express REST API
│   │   ├── package.json
│   │   ├── tsconfig.json
│   │   ├── src/
│   │   │   ├── index.ts              # Express server entrypoint
│   │   │   ├── fabric/               # Fabric Gateway connection
│   │   │   │   ├── gateway.ts        # gRPC + Gateway setup
│   │   │   │   ├── identity.ts       # Wallet & identity management
│   │   │   │   └── contracts.ts      # Contract accessor helpers
│   │   │   ├── routes/               # REST API routes
│   │   │   │   ├── auth.ts           # Authentication (JWT + Fabric identity)
│   │   │   │   ├── devices.ts        # Device registration/management
│   │   │   │   ├── guarantees.ts     # GO CRUD operations
│   │   │   │   ├── transfers.ts      # GO transfer operations
│   │   │   │   ├── conversions.ts    # Conversion (e→h) operations
│   │   │   │   ├── cancellations.ts  # Cancellation/claim operations
│   │   │   │   └── queries.ts        # Read-only query endpoints
│   │   │   ├── middleware/
│   │   │   │   ├── auth.ts           # JWT verification middleware
│   │   │   │   └── error.ts          # Error handling middleware
│   │   │   └── types/                # Shared TypeScript types
│   │   │       └── index.ts
│   │   └── tests/
│   └── frontend/                     # React SPA
│       ├── package.json
│       ├── tsconfig.json
│       ├── vite.config.ts
│       ├── index.html
│       └── src/
│           ├── main.tsx
│           ├── App.tsx
│           ├── api/                   # API client (axios/fetch)
│           │   └── client.ts
│           ├── components/
│           │   ├── Layout.tsx
│           │   ├── Dashboard/
│           │   │   ├── IssuerDashboard.tsx
```

```
│           │   │   ├── ProducerDashboard.tsx
│           │   │   └── ConsumerDashboard.tsx
│           │   ├── Devices/
│           │   │   ├── DeviceList.tsx
│           │   │   └── RegisterDevice.tsx
│           │   ├── Guarantees/
│           │   │   ├── GOList.tsx
│           │   │   ├── GODetail.tsx
│           │   │   ├── CreateGO.tsx
│           │   │   └── TransferGO.tsx
│           │   ├── Conversion/
│           │   │   ├── BacklogView.tsx
│           │   │   └── ConvertGO.tsx
│           │   ├── Certificates/
│           │   │   ├── CancellationList.tsx
│           │   │   └── CancelGO.tsx
│           │   └── common/
│           │       ├── Table.tsx
│           │       ├── Modal.tsx
│           │       └── StatusBadge.tsx
│           ├── hooks/
│           │   └── useAuth.ts
│           ├── pages/
│           │   ├── Login.tsx
│           │   ├── Dashboard.tsx
│           │   ├── Devices.tsx
│           │   ├── Guarantees.tsx
│           │   ├── Transfers.tsx
│           │   └── Certificates.tsx
│           └── styles/
│               └── globals.css
│
├── testing/                          # Retained: Caliper benchmarks
│   ├── caliper-config-GO/
│   └── Test-results/
│
└── version1/                         # PRESERVED: Original prototype (read-only reference)
```

---

# 3. Tiered Network Architecture

## 3.1 Organization Roles

Instead of hardcoded organizations (buyer, eproducer, hproducer, issuer), the new network uses **role-based** organizations. Any number of organizations can join the network under one of three tiers:

| Tier | Role | MSP Convention | Description |
|---|---|---|---|
| **Tier 1** | Issuer | `issuer<N>MSP` | Registry operator. Oversees device registration, GO validation, auditing. At least one issuer is required. |
| **Tier 2** | Producer | `producer<N>MSP` | Energy producer. Can register multiple device types (electricity meters, hydrogen output meters, etc.). Each producer can produce GOs for **any carrier** they have registered devices for. |
| **Tier 3** | Consumer | `consumer<N>MSP` | Energy buyer/consumer. Can receive GOs via transfer, cancel GOs to claim renewable attributes. |

## 3.2 Channel Policy

```
# Endorsement: Issuer + at least one other org
Endorsement:
  Type: Signature
  Rule: "OutOf(2, 'issuer1MSP.peer', 'producer1MSP.peer', 'consumer1MSP.peer')"
```

For operations that modify GOs:

- **Issuance**: Requires endorsement from the producer's peer AND the issuer's peer
- **Transfer**: Requires endorsement from the sender's peer AND one other org (e.g., issuer)
- **Cancellation**: Requires endorsement from the cancelling org's peer AND the issuer

## 3.3 Private Data Collections (Dynamic)

Collections follow the naming convention `privateDetails-{OrgMSPID}`:

```
[
  {
    "name": "publicGOcollection",
    "policy": "OR(<all-org-members>)",
    "requiredPeerCount": 0,
    "maxPeerCount": 3,
    "blockToLive": 0,
    "memberOnlyRead": true,
    "memberOnlyWrite": true
  },
  {
    "name": "privateDetails-issuer1MSP",
    "policy": "OR('issuer1MSP.member')",
    "requiredPeerCount": 0,
    "maxPeerCount": 1,
    "blockToLive": 0,
    "memberOnlyRead": true,
    "memberOnlyWrite": false,
    "endorsementPolicy": {
      "signaturePolicy": "OR('issuer1MSP.member')"
    }
  },
  {
    "name": "privateDetails-producer1MSP",
    "policy": "OR('producer1MSP.member', 'issuer1MSP.member')",
    "requiredPeerCount": 0,
    "maxPeerCount": 2,
    "blockToLive": 0,
    "memberOnlyRead": true,
    "memberOnlyWrite": false,
    "endorsementPolicy": {
      "signaturePolicy": "OR('producer1MSP.member', 'issuer1MSP.member')"
    }
  }
]
```

When a new org joins the network, a new `privateDetails-{newOrgMSP}` collection is added via chaincode upgrade.

### 3.4 Device Registration (New Feature)

Devices (SmartMeters, OutputMeters) are now **on-chain assets**:

```
type Device struct {
    DeviceID       string   `json:"deviceID"`
    DeviceType     string   `json:"deviceType"`     // "SmartMeter", "OutputMeter"
    OwnerOrgMSP    string   `json:"ownerOrgMSP"`
    EnergyCarriers []string `json:"energyCarriers"`  // ["electricity"], ["hydrogen"], or
both
    Status         string   `json:"status"`          // "active", "revoked", "suspended"
    RegisteredBy   string   `json:"registeredBy"`    // Issuer identity
    RegisteredAt   int64    `json:"registeredAt"`
    Attributes     map[string]string `json:"attributes"` // maxEfficiency,
emissionIntensity, etc.
}
```

This replaces the X.509 ABAC attributes for device validation. Benefits:

- Devices can be registered/revoked at runtime without re-enrolling certificates
- Multiple devices per organization
- Flexible attribute management

---

# 4. Chaincode Architecture (Object-Oriented)

## 4.1 Asset Hierarchy

```
BaseGO (interface)
├── ElectricityGO           — electricity guarantee
├── HydrogenGO              — hydrogen guarantee
└── (extensible: BiogasGO, HeatGO, ...)

Device                      — metering device registration
├── SmartMeter              — electricity production meter
└── OutputMeter             — hydrogen/other output meter

Certificate (interface)
├── CancellationStatement   — proof of GO cancellation
└── ConsumptionDeclaration  — record of energy input consumption
```

## 4.2 Base GO Interface

```
// assets/base.go
type BaseGOPublic struct {
    AssetID          string `json:"assetID"`
    CreationDateTime int64  `json:"creationDateTime"`
    GOType           string `json:"goType"`           // "electricity", "hydrogen", etc.
    ProducerOrgMSP   string `json:"producerOrgMSP"`
}

type BaseGOPrivate struct {
    AssetID                 string   `json:"assetID"`
    OwnerID                 string   `json:"ownerID"`
    CreationDateTime        int64    `json:"creationDateTime"`
    Quantity                float64  `json:"quantity"`         // MWh for electricity, kg
for hydrogen
    QuantityUnit            string   `json:"quantityUnit"`     // "MWh", "kg"
    Emissions               float64  `json:"emissions"`
    ProductionMethod        string   `json:"productionMethod"`
    ConsumptionDeclarations []string `json:"consumptionDeclarations"`
    DeviceID                string   `json:"deviceID"`         // Which device produced this
}
```

### 4.3 Contract Registration (main.go)

```
func main() {
    issuanceContract := new(contracts.IssuanceContract)
    issuanceContract.Name = "issuance"

    transferContract := new(contracts.TransferContract)
    transferContract.Name = "transfer"

    conversionContract := new(contracts.ConversionContract)
    conversionContract.Name = "conversion"

    cancellationContract := new(contracts.CancellationContract)
    cancellationContract.Name = "cancellation"

    queryContract := new(contracts.QueryContract)
    queryContract.Name = "query"

    deviceContract := new(contracts.DeviceContract)
    deviceContract.Name = "device"

    chaincode, err := contractapi.NewChaincode(
        issuanceContract,
        transferContract,
        conversionContract,
        cancellationContract,
        queryContract,
        deviceContract,
    )
    // ...
}
```

Clients invoke functions as `issuance:CreateElectricityGO`, `transfer:TransferGO`, etc.

### 4.4 ID Generation (v3.0 — Hash-Based Deterministic IDs)

v2.0 used on-chain sequential counters (`GetNextID`) that created a hot-key serialization bottleneck (MVCC_READ_CONFLICT). v3.0 replaced this with deterministic hash-based IDs:

```go
// assets/counter.go — v3.0
func GenerateID(ctx contractapi.TransactionContextInterface, prefix string, suffix int)
(string, error) {
    txID := ctx.GetStub().GetTxID()
    raw := txID + "_" + strconv.Itoa(suffix)
    hash := sha256.Sum256([]byte(raw))
    return prefix + hex.EncodeToString(hash[:8]), nil
}
```

Each transaction ID is guaranteed unique by Fabric. The `suffix` parameter disambiguates multiple IDs within one transaction (e.g., cancellation creates a certificate + remainder GO). This approach:

- **Eliminates shared state**: No `GetState` / `PutState` on a shared counter key
- **Enables parallel writes**: Every transaction's read-write set is independent
- **Is deterministic**: All endorsing peers compute the same ID from the same txID
- **Is collision-resistant**: SHA-256 with 8-byte (64-bit) output gives negligible collision probability for practical volumes

Prefix constants (`PrefixDevice = "device_"`, `PrefixEGO = "eGO_"`, etc.) and range-end constants (`RangeEndDevice = "device_~"`) ensure consistent key formatting and efficient range queries.

The legacy `GetNextID()` function is retained but marked DEPRECATED for backward compatibility.

**Impact:** Write throughput improved from 0.8 TPS to 50.5 TPS (63× improvement) with 100% success rate.

## 4.5 Role-Based Access Control

```go
// access/roles.go
func GetOrgRole(ctx contractapi.TransactionContextInterface) (string, error) {
    mspID, err := ctx.GetClientIdentity().GetMSPID()
    if err != nil {
        return "", err
    }
    // Determine role from on-chain org registry or MSP naming convention
    role, err := ctx.GetStub().GetState("orgRole_" + mspID)
    if err != nil || role == nil {
        return "", fmt.Errorf("organization %s is not registered", mspID)
    }
    return string(role), nil
}

func RequireRole(ctx contractapi.TransactionContextInterface, requiredRole string) error {
    role, err := GetOrgRole(ctx)
    if err != nil {
        return err
    }
    if role != requiredRole {
        return fmt.Errorf("access denied: requires role %s, got %s", requiredRole, role)
    }
    return nil
}

func GetCollectionName(ctx contractapi.TransactionContextInterface) (string, error) {
    mspID, err := ctx.GetClientIdentity().GetMSPID()
    if err != nil {
        return "", err
    }
    return "privateDetails-" + mspID, nil
}
```

### 4.6 Bug Fixes Summary

| # | Bug | Fix |
|---|---|---|
| 1 | In-memory counters reset on restart → ID collisions | On-chain persistent counters (§4.4) |
| 2 | Race condition in counter (read after unlock) | Fabric deterministic execution, single PutState per tx |
| 3 | IssuehGO overwrites accumulated emissions at end | Track emissions in running total; don't overwrite |
| 4 | IssuehGO skips final eGO (no else branch) | Add else clause for `AmountMWh < backlog.UsedMWh` |
| 5 | QueryPrivateeGOs remove() loop skips elements | Iterate backwards or use copy-on-write slice |
| 6 | No bounds checking → panics | Validate all inputs at function entry |
| 7 | `organization` attr vs `GetMSPID()` inconsistency | Use `GetMSPID()` exclusively for collection names |
| 8 | Remainder eGO CreationDateTime reset | Preserve original CreationDateTime on splits |
| 9 | ConsumptionDeclarations duplicated on hGO splits | Deep-copy declarations, only assign relevant ones |
| 10 | ConsumptionDeclarationHydrogen.DateTime is string | Unify to `int64` across all types |
| 11 | VerifyCancellationStatement hash never matches | Fix hash comparison to use correct key format |
| 12 | No access control on read functions | Add role-based read checks where appropriate |

---

# 5. Frontend Architecture

## 5.1 Technology Stack

| Layer | Technology | Purpose |
|---|---|---|
| Frontend | React 18 + TypeScript | Single-page application |
| Styling | Tailwind CSS | Utility-first CSS |
| Build | Vite | Fast development server |
| Backend | Express.js + TypeScript | REST API layer |
| Fabric SDK | `@hyperledger/fabric-gateway` | Blockchain interaction |
| Auth | JWT tokens | Session management |

## 5.2 Backend API Routes

```
POST   /api/auth/login              # Authenticate with Fabric identity
POST   /api/auth/enroll             # Enroll new user via Fabric CA

POST   /api/devices                 # Register a new device
GET    /api/devices                 # List devices for current org
GET    /api/devices/:id             # Get device details
PUT    /api/devices/:id/revoke      # Revoke a device

POST   /api/guarantees/electricity  # Create electricity GO (from SmartMeter)
POST   /api/guarantees/hydrogen     # Create hydrogen GO (from OutputMeter)
GET    /api/guarantees              # List GOs for current org
GET    /api/guarantees/:id          # Get GO details (public)
GET    /api/guarantees/:id/private  # Get GO private details

POST   /api/transfers               # Transfer GO to another org
POST   /api/transfers/by-amount     # Transfer partial GO by amount

POST   /api/conversions/backlog     # Add to hydrogen backlog
POST   /api/conversions/issue       # Issue hydrogen GO from backlog

POST   /api/cancellations/electricity  # Cancel electricity GO (claim attributes)
POST   /api/cancellations/hydrogen     # Cancel hydrogen GO (claim attributes)
GET    /api/cancellations              # List cancellation statements
POST   /api/cancellations/verify       # Verify cancellation statement

GET    /api/queries/ego-list        # Get all current electricity GOs
GET    /api/queries/hgo-list        # Get all current hydrogen GOs
```

## 5.3 Frontend Pages

| Page | Role | Description |
|---|---|---|
| **Login** | All | Identity selection / enrollment |
| **Dashboard** | All | Role-specific overview with key metrics |
| **Devices** | Issuer, Producer | Register/manage metering devices |
| **My GOs** | Producer, Consumer | View owned GOs, initiate transfers |
| **Transfer** | Producer, Consumer | Transfer GOs between orgs |
| **Conversion** | Producer | Manage hydrogen backlog, issue converted GOs |
| **Certificates** | All | View cancellation statements and consumption declarations |
| **Audit** | Issuer | Cross-org GO tracking and verification |

## 5.4 Gateway Connection Pattern

```
// backend/src/fabric/gateway.ts
import { connect, Contract, Identity, Signer, signers } from '@hyperledger/fabric-gateway';
import * as grpc from '@grpc/grpc-js';

export async function connectGateway(orgMSP: string, certPath: string, keyPath: string) {
    const tlsRootCert = await fs.readFile(tlsCertPath);
    const tlsCredentials = grpc.credentials.createSsl(tlsRootCert);
    const client = new grpc.Client(peerEndpoint, tlsCredentials, {
        'grpc.ssl_target_name_override': peerHostOverride,
    });

    const credentials = await fs.readFile(certPath);
    const identity: Identity = { mspId: orgMSP, credentials };

    const privateKeyPem = await fs.readFile(keyPath);
    const privateKey = crypto.createPrivateKey(privateKeyPem);
    const signer: Signer = signers.newPrivateKeySigner(privateKey);

---

## 6. v4.0 — Hardening (Scalability, Privacy, Auditability)

v4.0 implements ADRs 006–010, addressing the critique's scalability, privacy, and audit
gaps.

### 6.1 Cursor-Based Pagination (ADR-006)
All list endpoints (`GetCurrentEGOsList`, `GetCurrentHGOsList`, `ListDevices`) now have
paginated variants using Fabric's `GetStateByRangeWithPagination`. Default page size: 50,
max: 200. Returns `PaginatedResult{Records, Bookmark, Count}` for cursor-based iteration.

### 6.2 Tombstone Pattern (ADR-007)
GOs are no longer physically deleted. The `Status` field on public GO structs tracks
lifecycle state: `active` → `cancelled` | `transferred`.
`DeleteEGOFromLedger`/`DeleteHGOFromLedger` now update Status instead of calling `DelState`.
Query list functions filter tombstoned records. A new CouchDB composite index on `[Status,
GOType]` supports filtered queries.

### 6.3 Timestamp Drift Guard (ADR-008)
`GetTimestamp()` validates that the proposal timestamp is within 300 seconds of the orderer
block time. Prevents backdating attacks where a malicious client submits a GO with a
manipulated creation timestamp.

### 6.4 Hash Commitment / Selective Disclosure (ADR-009)
Each GO stores a `QuantityCommitment = SHA-256(quantity || salt)` on the public ledger. The
`CommitmentSalt` is stored in private data. Producers can selectively disclose their
quantity to verifiers by revealing the salt, enabling third-party verification without
```

collection access.

### 6.5 CouchDB Hardening (ADR-010)
Private data collections now have `blockToLive: 1000000` (~70 days at 10 blocks/min). This prevents unbounded CouchDB growth while retaining data for the audit retention period. The `publicGOcollection` retains `blockToLive: 0` (permanent).

---

## 7. v5.0 — Interoperability & Multi-Carrier

v5.0 implements ADRs 012–016, addressing the critique's interoperability and extensibility gaps.

### 7.1 CEN-EN 16325 Data Model (ADR-012)
Public GO structs now include standard-aligned fields: `CountryOfOrigin` (ISO 3166-1), `GridConnectionPoint` (EIC code), `SupportScheme`, `EnergySource` (EN 16325 code), `ProductionPeriodStart/End`. All fields use `omitempty` for backward compatibility.

### 7.2 API Versioning (ADR-013)
New `AdminContract` exposes `GetVersion()` returning `VersionInfo{Version, ChaincodeID, SupportedAPIs, BreakingChange}`. Clients call this before invoking domain functions to verify compatibility. Supported APIs follow `<contract>/v1` naming.

### 7.3 Dynamic Org Onboarding (ADR-014)
`RegisterOrganization` (issuer-only) records organization metadata on-chain: MSP, display name, type, energy carriers, country. This provides the application-layer onboarding record. Fabric channel config updates remain an out-of-band admin operation.

### 7.4 Biogas Carrier (ADR-015)
New `BiogasContract` with `CreateBiogasGO` and `CancelBiogasGO`. Biogas-specific attributes: `VolumeNm3`, `EnergyContentMWh`, `BiogasProductionMethod`, `FeedstockType`. Demonstrates the platform's carrier-extensibility per RED III. Includes full v4.0 patterns (Status, commitment, tombstone).

### 7.5 Event-Driven CQRS (ADR-016)
`EmitLifecycleEvent` helper emits Fabric chaincode events (JSON payload) for key operations: `GO_CREATED`, `GO_TRANSFERRED`, `GO_CANCELLED`, `GO_CONVERTED`, `GO_SPLIT`, `DEVICE_REGISTERED`, `DEVICE_REVOKED`. Off-chain listeners consume these to build query-optimised read models.

### 7.6 Contract Registry (v5.0)

| # | Namespace | Key Functions |
|---|---|---|
| 1 | `issuance` | CreateElectricityGO, CreateHydrogenGO |
| 2 | `transfer` | TransferEGO, TransferEGOByAmount, TransferHGOByAmount |
| 3 | `conversion` | AddHydrogenToBacklog, IssuehGO |
| 4 | `cancellation` | ClaimRenewableAttributesElectricity/Hydrogen, |

```
VerifyCancellationStatement |
| 5 | `query` | 18 functions: point reads, paginated lists, commitment verification, biogas
queries |
| 6 | `device` | RegisterDevice, GetDevice, ListDevices(Paginated),
Revoke/Suspend/Reactivate |
| 7 | `admin` | GetVersion, RegisterOrganization, GetOrganization |
| 8 | `biogas` | CreateBiogasGO, CancelBiogasGO |

---

## 8. v6.0 — Production Hardening

v6.0 implements ADRs 017–022, focusing on cryptographic hardening, standards compliance, and
deprecation management.

### 8.1 Cryptographically Secure Commitment Salts (ADR-017)

`GenerateCommitment()` now uses `crypto/rand` to produce 128-bit random salts instead of
deriving salts from `SHA-256(txID)`. The previous approach was brute-forceable since `txID`
is public. The new scheme stores the random salt exclusively in private data collections,
making the quantity commitment `SHA-256(quantity || random_salt)` computationally secure.

### 8.2 CEN-EN 16325 Field Validation (ADR-018)

New validation module `util/validate_cen.go` enforces European GO standard compliance at
write time:
- `ValidateCountryOfOrigin`: ISO 3166-1 alpha-2 against 30 EU/EEA/CH/GB codes
- `ValidateEnergySource`: EECS fact sheet regex `^F\d{8}$`
- `ValidateSupportScheme`: Enumerated CEN-EN 16325 categories (FIT, FIP, quota, tax, loan,
none, other)
- `ValidateGridConnectionPoint`: 16-character EIC code regex
- `ValidateProductionPeriod`: `end > start`, maximum 366 days

### 8.3 State-Based Endorsement (ADR-019)

After GO creation, `SetStateValidationParameter()` binds the asset's endorsement policy to
the producing organization and issuer. This replaces the channel-level majority policy for
per-asset write operations, preventing unauthorized modification by non-owning
organizations.

### 8.4 Deprecation Policy (ADR-021/022)

Formal deprecation lifecycle: deprecation warnings in response metadata → client migration
period → eventual removal. Applied to unpaginated query functions (`GetCurrentEGOsList`,
`GetCurrentHGOsList`, `ListDevices`) which return warnings directing clients to paginated
variants.

---
```

```
## 9. v7.0 — Market Integration

v7.0 implements ADRs 024, 027, and 029, extending the platform with cross-registry bridging,
IoT device attestation, and external data oracle capabilities.

### 9.1 Cross-Registry Bridge (ADR-024)

New `BridgeContract` manages GO movements between registries:

```go
type BridgeTransfer struct {
    TransferID       string  `json:"transferId"`
    Direction        string  `json:"direction"`        // "export" or "import"
    GOAssetID        string  `json:"goAssetId"`
    ExternalRegistry string  `json:"externalRegistry"` // e.g. "AIB-hub", "NECS"
    ExternalID       string  `json:"externalId"`
    GOType           string  `json:"goType"`
    Status           string  `json:"status"`           // "pending", "confirmed", "failed"
    CountryOfOrigin  string  `json:"countryOfOrigin,omitempty"`
    EnergySource     string  `json:"energySource,omitempty"`
}
```
```

Functions:

- `ExportGO`: Locks source GO (status → "transferred"), creates pending bridge record
- `ConfirmExport`: Transitions pending → confirmed on destination acknowledgment
- `ImportGO`: Creates local GO asset from external registry data, bridge record auto-confirmed
- `GetBridgeTransfer`, `ListBridgeTransfersPaginated`: Read operations with pagination

## 9.2 IoT Device Attestation (ADR-027)

ECDSA P-256 signature verification for metering device readings:

- **Device struct extended**: `PublicKeyPEM` field (optional, backward-compatible) stores the device's ECDSA public key
- `VerifyDeviceReading(deviceID, readingJSON, signatureBase64)`: Verifies an ECDSA signature against the device's registered public key without writing state
- `SubmitSignedReading(deviceID, readingJSON, signatureBase64)`: Verifies signature, then stores the verified reading on-chain

This creates a cryptographic chain from physical smart meter → on-chain record, assuming the meter holds its private key in a tamper-resistant HSM.

## 9.3 External Data Oracle (ADR-029)

New `OracleContract` for ENTSO-E grid generation data:

```
type GridGenerationRecord struct {
    RecordID       string  `json:"recordId"`
    BiddingZone    string  `json:"biddingZone"`     // e.g. "DE-LU", "NL"
    PeriodStart    int64   `json:"periodStart"`
    PeriodEnd      int64   `json:"periodEnd"`
    EnergySource   string  `json:"energySource"`    // EECS code
    GenerationMW   float64 `json:"generationMW"`
    EmissionFactor float64 `json:"emissionFactor"`  // gCO2eq/kWh
    DataSource     string  `json:"dataSource"`      // "ENTSO-E-TP", "ENTSOG"
}
```

Functions:

- `PublishGridData`: Issuer-only; writes grid generation records from transient data
- `GetGridData`, `ListGridDataPaginated`: Read operations
- `CrossReferenceGO`: Validates a GO's production period and energy source against oracle data

## 9.4 Contract Registry (v7.0)

| # | Namespace | Key Functions |
|---|---|---|
| 1 | `issuance` | CreateElectricityGO, CreateHydrogenGO |
| 2 | `transfer` | TransferEGO, TransferEGOByAmount, TransferHGOByAmount |
| 3 | `conversion` | AddHydrogenToBacklog, IssuehGO |
| 4 | `cancellation` | ClaimRenewableAttributesElectricity/Hydrogen, VerifyCancellationStatement |
| 5 | `query` | 18+ functions: point reads, paginated lists, commitment verification, deprecation warnings |
| 6 | `device` | RegisterDevice, GetDevice, ListDevices(Paginated), Revoke/Suspend/Reactivate, VerifyDeviceReading, SubmitSignedReading |
| 7 | `admin` | GetVersion (7.0.0), RegisterOrganization, GetOrganization |
| 8 | `biogas` | CreateBiogasGO, CancelBiogasGO |
| 9 | `bridge` | ExportGO, ConfirmExport, ImportGO, GetBridgeTransfer, ListBridgeTransfersPaginated |
| 10 | `oracle` | PublishGridData, GetGridData, ListGridDataPaginated, CrossReferenceGO |

**Supported APIs (v7.0.0):** issuance/v1, transfer/v1, conversion/v1, cancellation/v1, query/v1, device/v1, admin/v1, bridge/v1, oracle/v1

### 9.5 Benchmark Validation (v7.0)

Caliper v0.6.0 (28 rounds, 10 workers, Hetzner 16 vCPU / 32 GB) with 901 registered devices:

| Metric | v3.0 | v5.0 | v7.0 |
|---|---|---|---|
| Write success rate | 100% | 100% | **100%** |
| Max write throughput | 50.5 TPS | 50.0 TPS | **50.4 TPS** |
| Point read throughput | >2,000 TPS | >2,000 TPS | **2,000 TPS** |
| Write latency (serial) | 0.10s | 0.10s | **0.10s** |
| GetDevice at 2,000 TPS | — | — | **0% failure** |
| Pagination (500 TPS) | — | 0% failure | **0% failure** |
| ListDevices unpaginated (901 devices) | — | — | **80% failure at 500 TPS** |

Detailed results: see `testing/PERFORMANCE_REPORT_v7.md`

```
return connect({ client, identity, signer, hash: hash.sha256 });
```

}

```
---

## 6. Implementation Phases

### Phase 1: Chaincode Restructure
1. Create the `chaincode/` directory structure
2. Define asset types in `assets/`
3. Implement persistent counters
4. Implement role-based access control in `access/`
5. Migrate all 26 exported functions into domain contracts
6. Fix all 12 bugs
7. Unit tests for each contract

### Phase 2: Network Configuration
1. Create new `network/` with tiered configtx.yaml
2. Create Docker Compose templates for each org role
3. Implement `add-org.sh` for dynamic org onboarding
4. Update collection-config.json for role-based collections
5. Test network bring-up with 1 issuer + 1 producer + 1 consumer

### Phase 3: Frontend Application
1. Backend: Express + Fabric Gateway connection
2. Backend: REST API routes for all chaincode functions
3. Frontend: React app scaffold with Vite
4. Frontend: Login/auth flow with Fabric identity
5. Frontend: Role-specific dashboards
6. Frontend: GO lifecycle pages (create, transfer, convert, cancel)
7. Frontend: Device management pages
8. Frontend: Certificate viewing and verification

### Phase 4: Integration & Testing
1. End-to-end tests (frontend → backend → chaincode)
2. Update Caliper benchmarks for new chaincode
3. Documentation updates
4. CI/CD pipeline setup

---

## 7. Migration Notes

- The original `version1/` directory is **preserved intact** as a reference
- The new `chaincode/` replaces `version1/artifacts/Mychaincode/`
- The new `network/` replaces `version1/setup1/` and `version1/artifacts/channel/`
- The new `collections/` replaces `version1/artifacts/private-data-collections/`
- All existing DOCS.md files remain in `version1/`

---
```

```
## 8. v3.0 Architecture Changes

### 8.1 Performance Optimizations

| Change | Description | File(s) |
|--------|-------------|--------|
| **Hash-based IDs (ADR-001)** | Replace `GetNextID()` sequential counter with
`GenerateID()` using SHA-256 of transaction ID | `assets/counter.go`, all contract files |
| **CouchDB Indexes (ADR-002)** | Composite indexes on `[OwnerID, GOType]` and `[GOType,
CreationDateTime]` | `chaincode/META-INF/statedb/couchdb/indexes/` |
| **BatchTimeout (ADR-005)** | Reduced from 2s to 500ms for lower write latency |
`network/configtx.yaml` |

### 8.2 Security Hardening

| Change | Description | File(s) |
|--------|-------------|--------|
| **Secure InitLedger (ADR-004)** | Caller MSP must match the `issuerMSP` argument —
prevents cross-org privilege escalation | `contracts/device_mgmt.go` |

### 8.3 Data Architecture — ID Format Change
```

v2.0: eGO1, eGO2, eGO3, ... (sequential, shared counter) v3.0: eGO_a1b2c3d4e5f6a7b8, ... (hash-based, contention-free)

```
Range queries use lexicographic bounds (e.g., `"eGO"` to `"eGO~"`) that capture both
formats, enabling mixed-format operation during migration.

### 8.4 Block Configuration

| Parameter | v2.0 | v3.0 | Rationale |
|-----------|------|------|----------|
| BatchTimeout | 2s | **500ms** | Reduces write latency floor by 75% |
| MaxMessageCount | 10 | 10 | Unchanged |
| PreferredMaxBytes | 512 KB | 512 KB | Unchanged |

### 8.5 Benchmark Validation

All changes validated by Hyperledger Caliper v0.6.0 on a 16-vCPU Hetzner VM:

| Metric | v2.0 | v3.0 |
|--------|------|------|
| Write success rate | 10% | **100%** |
| Max write throughput | 0.8 TPS | **50.5 TPS** |
| Point read throughput | >2,000 TPS | >2,000 TPS |
| Write latency (serial) | 2.09s | **0.10s** |

Detailed results: see `testing/PERFORMANCE_REPORT.md`
```